LAB EXPERIMENTS

QUESTION NO -1:

Aim

To develop a Python program to represent the distribution of grades obtained by 40 students using a pie chart and a bar chart, identify the maximum and minimum grades by student count, and calculate the combined percentage of students who secured grades A and S.

Algorithm

1. Start the program.
2. Store the grades F, E, D, C, B, A and S in a list.
3. Store the corresponding number of students as 3, 4, 5, 8, 9, 7 and 4.
4. Calculate the total number of students and the percentage for each grade.
5. Draw a pie chart showing the percentage distribution of grades.
6. Display grade labels and percentage values on the pie chart.
7. Draw a bar chart comparing the number of students in each grade.
8. Label the X-axis as Grades and the Y-axis as Number of Students.
9. Display the exact number of students above each bar.
10. Identify the grades with the maximum and minimum number of students.
11. Calculate the combined percentage of students who secured grades A and S.
12. Display the results.
13. Stop the program.

CODE:

```
import matplotlib.pyplot as plt

grades = ['F', 'E', 'D', 'C', 'B', 'A', 'S']

students = [3, 4, 5, 8, 9, 7, 4]

= sum(students)

print("Total number of students:", total)

print("\nGrade-wise student details:")

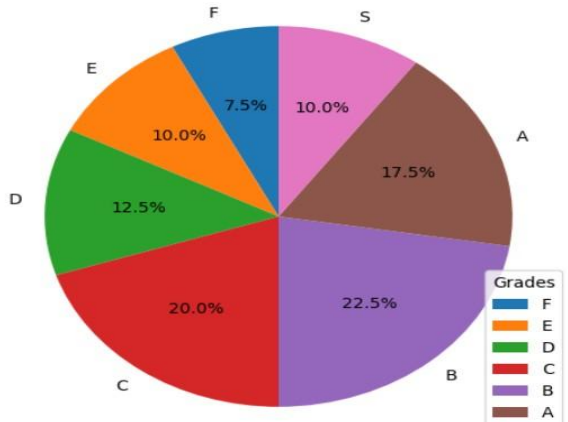
 i in range(len(grades)):
```

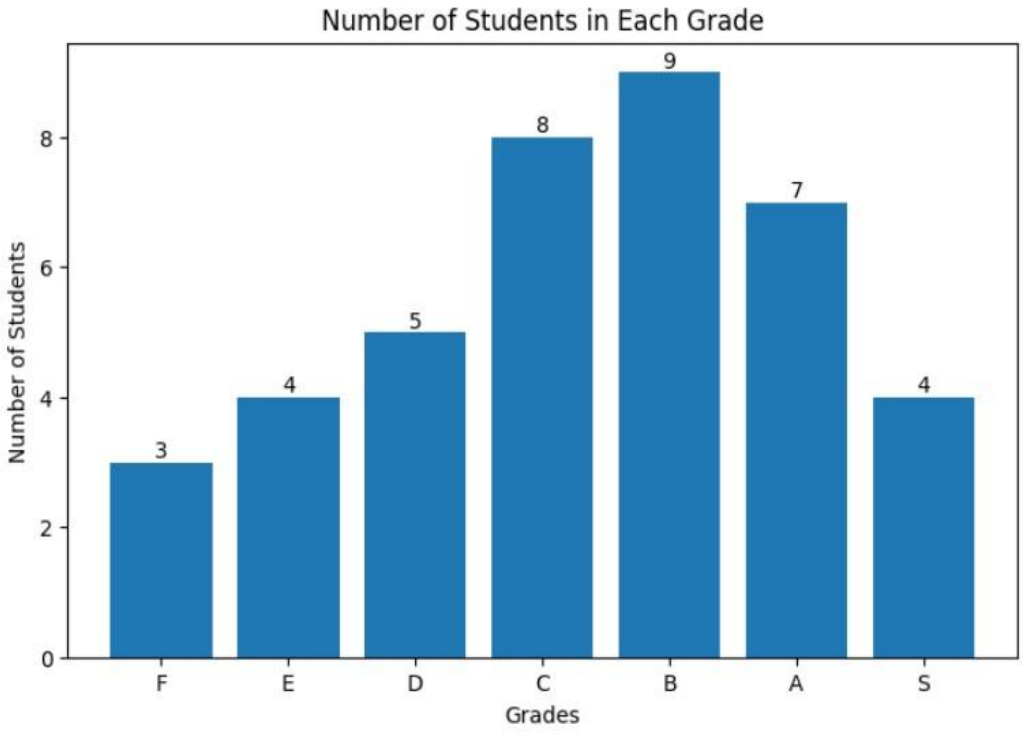
```
    percentage = (students[i] / total) * 100
print(grades[i], ":", students[i],
          "students -", round(percentage, 2), "%")
plt.figure(figsize=(7,
6)) plt.pie(     students,
labels=grades,
autopct='%1.1f%%',
startangle=90
)
plt.title("Percentage Distribution of Student Grades")
plt.legend(grades, title="Grades") plt.show()
plt.figure(figsize=(8, 5)) bars =
plt.bar(grades, students)
plt.xlabel("Grades")
plt.ylabel("Number of
Students") plt.title("Number of
Students in Each Grade")
plt.show()
print("\nMaximum students:", max_grades, "-", max_count) print("Minimum
students:", min_grades, "-", min_count)
combined = students[5] + students[6] percentage
= (combined / total) * 100
print("Students with grades A and S:", combined)
print("Combined percentage of A and S:",
round(percentage, 2), "%")
```

Expected output :

```
Total number of students: 40

Grade-wise student details:
F : 3 students - 7.5 %
E : 4 students - 10.0 %
D : 5 students - 12.5 %
C : 8 students - 20.0 %
B : 9 students - 22.5 %
A : 7 students - 17.5 %
S : 4 students - 10.0 %
```






```
Maximum students: ['B'] - 9
Minimum students: ['F'] - 3
Students with grades A and S: 11
Combined percentage of A and S: 27.5 %
```

Question No:2
2) Semantic Network Using NetworkX

Aim: To represent concepts and labeled relationships as a directed semantic network.
Software Required: Python 3.x and NetworkX (install with: pip install networkx)

Algorithm: Semantic Network Using NetworkX

1. Start.
2. Import the networkx library.
3. Create a directed graph.
4. Add concepts as nodes through edges.
5. Add relationships between concepts, such as is-a and has-instance.
6. Display each relationship between the concepts.
7. Draw the semantic network graph.
8. Display the graph with labeled nodes and edges.
9. Stop.

Program:

```
!pip install networkx matplotlib
import networkx as nx
import matplotlib.pyplot as plt
# Create a directed graph


G = nx.DiGraph()
# Add concepts and relationships
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")

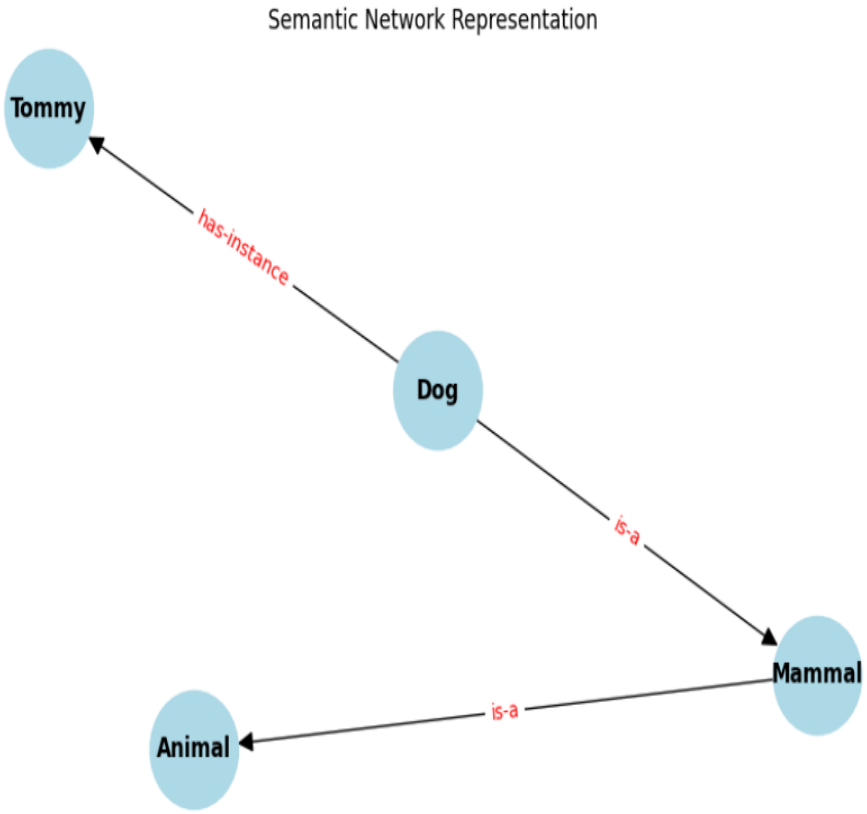
# Display relationships
print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
    print(source, "--", data["relation"], "-->", target)

# Draw the semantic network
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_color="lightblue",
        node_size=2500, font_size=10, arrows=True)
```

```
edge_labels = nx.get_edge_attributes(G, "relation")
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
plt.title("Semantic Network")
plt.show()
```

OUTPUT:

```
Semantic Network Relationships:
Dog -- is-a --> Mammal
Dog -- has-instance --> Tommy
Mammal -- is-a --> Animal
```



Question No:3

3)Frame-Based Knowledge Representation

Aim: To represent an object as a frame containing slots and values.

Software Required: Python 3.x

Algorithm: Frame-Based Knowledge Representation

1. Start.
2. Create a dictionary to represent a vehicle frame.
3. Store the vehicle details as slots and values.
4. Include slots such as class, brand, fuel type, wheels, and color.
5. Display the title "Vehicle Frame".
6. Use a loop to access each slot and its corresponding value.
7. Print all the slots and values.
8. Stop.

CODE:

```
# Create a vehicle frame using a dictionary
vehicle = {
    "class": "Vehicle",
    "brand": "Toyota",
    "fuel_type": "Petrol",
    "wheels": 4,
    "color": "White"
}
# Display the frame
print("Vehicle Frame")
for slot, value in vehicle.items():
    print(slot, ":", value)
```

OUTPUT:

```
Vehicle Frame
class : Vehicle
brand : Toyota
fuel_type : Petrol
wheels : 4
color : White
```

Frame-Based Knowledge Representation: Vehicle

Slot	Value
class	Vehicle
brand	Toyota
fuel_type	Petrol
wheels	4
color	White